HealthPal 后端技术框架详解

文档版本： v1.1

更新时间： 2026-03-10

作者： 全栈工程师 Agent

一、架构概述

1.1 系统架构图

```
graph TB
    subgraph Client["客户端层 (Client Layer)"]
        iOS["iOS App<br/>Flutter"]
        Android["Android App<br/>Flutter"]
        Web["Web 管理台<br/>Vue3/React"]
    end

    subgraph Access["接入层 (Access Layer)"]
        Nginx["Nginx 反向代理<br/>SSL 终止 / 负载均衡 / 缓存"]
    end

    subgraph Gateway["API 网关层 (API Gateway)"]
        FastAPI["FastAPI Application<br/>Uvicorn ASGI"]
        AuthMW["认证中间件"]
        RateMW["限流中间件"]
        LogMW["日志中间件"]
        CORSMW["CORS 中间件"]
    end

    subgraph App["应用服务层 (Application Layer)"]
        AuthService["用户服务<br/>AuthService"]
        RecordService["档案服务<br/>RecordService"]
        IndicatorService["指标服务<br/>IndicatorSvc"]
        MedService["用药服务<br/>MedService"]
        OCRService["OCR 服务<br/>OCRService"]
        AIService["AI 分析服务<br/>AIService"]
        NotifyService["通知服务<br/>NotifyService"]
```

```
        FileService["文件服务<br/>FileService"]
    end

    subgraph Data["数据持久层 (Data Persistence)"]
        MySQL["MySQL 8.0<br/>关系型数据"]
        MongoDB["MongoDB 6.0<br/>文档型数据"]
        Redis["Redis 7.0<br/>缓存/队列"]
        OSS["对象存储<br/>文件存储"]
    end

    iOS -->|HTTPS TLS 1.3| Nginx
    Android -->|HTTPS TLS 1.3| Nginx
    Web -->|HTTPS TLS 1.3| Nginx

    Nginx --> FastAPI
    FastAPI --> AuthMW
    FastAPI --> RateMW
    FastAPI --> LogMW
    FastAPI --> CORSMW

    FastAPI --> AuthService
    FastAPI --> RecordService
    FastAPI --> IndicatorService
    FastAPI --> MedService
    FastAPI --> OCRService
    FastAPI --> AIService
    FastAPI --> NotifyService
    FastAPI --> FileService

    AuthService --> MySQL
    RecordService --> MySQL
    IndicatorService --> MySQL
    MedService --> MySQL
    AIService --> MongoDB
    NotifyService --> Redis
    FileService --> OSS

    style Client fill:#e1f5ff
    style Access fill:#fff4e1
    style Gateway fill:#f0e1ff
    style App fill:#e1ffe1
    style Data fill:#ffe1e1
```

1.2 技术选型理由

技术	选型	理由
Web 框架	FastAPI	高性能异步、自动 OpenAPI 文档、类型安全
ASGI 服务器	Uvicorn	原生 ASGI 支持、高性能、热重载
ORM	SQLAlchemy 2.0	成熟稳定、支持异步、强大的查询能力
数据验证	Pydantic v2	与 FastAPI 深度集成、性能优秀
任务队列	Celery + Redis	成熟可靠、支持定时任务、分布式
缓存	Redis	高性能、丰富数据结构、发布订阅

二、项目目录结构

2.1 完整目录树

```
src/backend/
├── app/
│   ├── __init__.py
│   ├── main.py                 # 应用入口，FastAPI 实例
│   ├── config.py               # 配置管理（环境变量、常量）
│   ├── dependencies.py         # 依赖注入（认证、权限等）
│   │
│   ├── api/                    # API 路由层
│   │   ├── __init__.py
│   │   ├── router.py           # 路由汇总
│   │   ├── auth.py             # 认证接口（登录/注册/登出）
│   │   ├── users.py            # 用户接口（资料管理）
│   │   ├── family.py           # 家庭成员接口
│   │   ├── records.py          # 健康档案接口
│   │   ├── indicators.py       # 健康指标接口
│   │   ├── medications.py      # 用药提醒接口
│   │   └── ai.py               # AI 分析接口
│   │
│   ├── models/                 # 数据模型层（SQLAlchemy）
```

```
│   │   ├── __init__.py
│   │   ├── base.py             # 基类模型
│   │   ├── user.py             # 用户模型
│   │   ├── family_member.py    # 家庭成员模型
│   │   ├── health_record.py    # 健康档案模型
│   │   ├── health_indicator.py # 健康指标模型
│   │   ├── medication.py       # 用药提醒模型
│   │   └── __pycache__/
│   │
│   ├── schemas/                # Pydantic Schema 层
│   │   ├── __init__.py
│   │   ├── user.py             # 用户相关 Schema
│   │   ├── family.py           # 家庭成员 Schema
│   │   ├── record.py           # 健康档案 Schema
│   │   ├── indicator.py        # 健康指标 Schema
│   │   ├── medication.py       # 用药提醒 Schema
│   │   ├── ai.py               # AI 分析 Schema
│   │   └── common.py           # 通用 Schema（响应/分页）
│   │
│   ├── services/               # 业务逻辑层
│   │   ├── __init__.py
│   │   ├── auth_service.py     # 认证业务逻辑
│   │   ├── user_service.py     # 用户业务逻辑
│   │   ├── record_service.py   # 档案业务逻辑
│   │   ├── indicator_service.py# 指标业务逻辑
│   │   ├── medication_service.py# 用药业务逻辑
│   │   ├── ocr_service.py      # OCR 识别服务
│   │   ├── ai_service.py       # AI 分析服务
│   │   ├── file_service.py     # 文件存储服务
│   │   └── notify_service.py   # 通知推送服务
│   │
│   ├── db/                     # 数据库层
│   │   ├── __init__.py
│   │   ├── session.py          # 数据库会话管理
│   │   ├── mysql.py            # MySQL 连接配置
│   │   ├── mongodb.py          # MongoDB 连接配置
│   │   ├── redis.py            # Redis 连接配置
│   │   └── init_db.py          # 数据库初始化脚本
│   │
│   ├── middleware/             # 中间件层
│   │   ├── __init__.py
│   │   ├── auth.py             # JWT 认证中间件
│   │   ├── ratelimit.py        # 限流中间件
│   │   ├── logger.py           # 日志中间件
│   │   └── cors.py             # CORS 中间件
│   │
```

```
│   ├── utils/                  # 工具函数层
│   │   ├── __init__.py
│   │   ├── jwt.py              # JWT 工具
│   │   ├── password.py         # 密码加密工具
│   │   ├── sms.py              # 短信发送工具
│   │   ├── email.py            # 邮件发送工具
│   │   ├── validator.py        # 数据验证工具
│   │   └── helpers.py          # 辅助函数
│   │
│   ├── tasks/                  # Celery 异步任务
│   │   ├── __init__.py
│   │   ├── celery_app.py       # Celery 配置
│   │   ├── ocr_tasks.py        # OCR 识别任务
│   │   ├── ai_tasks.py         # AI 分析任务
│   │   └── notify_tasks.py     # 通知推送任务
│   │
│   └── exceptions/             # 异常处理
│       ├── __init__.py
│       └── handlers.py         # 全局异常处理器
│
├── tests/                      # 测试目录
│   ├── __init__.py
│   ├── conftest.py             # pytest 配置
│   ├── test_auth.py            # 认证测试
│   ├── test_users.py           # 用户接口测试
│   ├── test_records.py         # 档案接口测试
│   └── test_services.py        # 服务层测试
│
├── scripts/                    # 脚本工具
│   ├── setup.sh                # 环境搭建脚本
│   ├── init_db.py              # 数据库初始化
│   ├── seed_data.py            # 测试数据填充
│   └── backup_db.sh            # 数据库备份
│
├── migrations/                 # 数据库迁移（Alembic）
│   ├── versions/               # 迁移版本文件
│   ├── env.py
│   └── script.py.mako
│
├── requirements.txt            # Python 依赖
├── requirements-dev.txt        # 开发环境依赖
├── pytest.ini                  # pytest 配置
├── .env.example                # 环境变量示例
├── Dockerfile                  # Docker 镜像
└── docker-compose.yml          # Docker 编排
```

三、核心业务流程

3.1 用户认证流程

```
sequenceDiagram
    participant Client as 客户端
    participant API as API 网关
    participant Auth as 认证服务
    participant MySQL as MySQL
    participant Redis as Redis

    Client->>API: POST /auth/register<br/>(phone, password, verify_code)
    API->>Redis: 验证短信验证码
    Redis-->>API: 验证码有效
    API->>Auth: 检查手机号是否已注册
    Auth->>MySQL: SELECT * FROM users WHERE phone=?
    MySQL-->>Auth: 返回查询结果
    Auth-->>API: 手机号未注册
    API->>Auth: 创建新用户
    Auth->>MySQL: INSERT INTO users (...)
    MySQL-->>Auth: 返回用户 ID
    Auth->>Auth: 密码 bcrypt 加密
    Auth-->>API: 返回用户 UUID
    API-->>Client: 注册成功 {uuid}

    Note over Client,Redis: 登录流程
    Client->>API: POST /auth/login<br/>(phone, password)
    API->>Auth: 验证用户
    Auth->>MySQL: 查询用户信息
    MySQL-->>Auth: 返回用户数据
    Auth->>Auth: 验证密码哈希
    Auth->>Auth: 生成 JWT Token
    Auth-->>API: 返回 Token + 用户信息
    API->>Redis: 存储 Session (TTL 7 天)
    API-->>Client: 登录成功 {token, user}
```

3.2 健康档案上传 + OCR 流程

```
flowchart TD
    Start([开始]) --> Upload[客户端上传文件]
    Upload --> Validate{文件验证}
    Validate -->|格式错误 | Reject[拒绝上传]
    Validate -->|验证通过 | CreateRecord[创建档案记录]
    CreateRecord --> UploadOSS[上传到 OSS]
    UploadOSS --> GetURL[获取文件 URL]
    GetURL --> UpdateDB[更新档案 file_urls]
    UpdateDB --> TriggerOCR[触发 OCR 异步任务]
    TriggerOCR --> Queue[Celery 任务队列]

    Queue --> OCRWorker[OCR Worker 取任务]
    OCRWorker --> CallOCR[调用百度/腾讯 OCR API]
    CallOCR --> ParseOCR{解析结果}
    ParseOCR -->|失败 | Retry{重试?}
    Retry -->|是 | CallOCR
    Retry -->|否 | MarkFailed[标记 OCR 失败]
    ParseOCR -->|成功 | ExtractData[提取结构化数据]
    ExtractData --> UpdateStatus[更新档案 OCR 状态]
    UpdateStatus --> SaveResult[保存 OCR 结果]
    SaveResult --> End([结束])

    Reject --> End
    MarkFailed --> End

    style Start fill:#52c41a
    style End fill:#52c41a
    style Reject fill:#ff4d4f
    style MarkFailed fill:#ff9c6e
```

3.3 健康指标趋势查询流程

```
sequenceDiagram
    participant Client as 客户端
    participant API as API 网关
    participant Indicator as 指标服务
    participant MySQL as MySQL
    participant Redis as Redis

    Client->>API: GET /indicators/trend?<br/>indicator_code=BP_HIGH&<br/>member_id=xxx&<
```

```
    API->>Redis: 查询缓存
    Redis-->>API: 缓存未命中
    API->>Indicator: 获取指标趋势
    Indicator->>MySQL: SELECT * FROM health_indicators<br/>WHERE user_id=? AND<br/>indic
    MySQL-->>Indicator: 返回指标数据
    Indicator->>Indicator: 计算统计值<br/>(avg, min, max)
    Indicator->>Redis: 缓存结果 (TTL 1 小时)
    Indicator-->>API: 返回趋势数据
    API-->>Client: 返回 JSON 响应
```

四、核心模块详解

4.1 配置管理 (config.py)

```
from pydantic_settings import BaseSettings
from functools import lru_cache
from typing import Optional

class Settings(BaseSettings):
    """应用配置，支持环境变量覆盖"""

    # 应用基础配置
    APP_NAME: str = "HealthPal API"
    APP_VERSION: str = "1.0.0"
    DEBUG: bool = False
    API_PREFIX: str = "/api/v1"

    # 服务器配置
    HOST: str = "0.0.0.0"
    PORT: int = 8000
    WORKERS: int = 4

    # 数据库配置 - MySQL
    MYSQL_HOST: str = "localhost"
    MYSQL_PORT: int = 3306
    MYSQL_USER: str = "healthpal"
    MYSQL_PASSWORD: str = "your_password"
    MYSQL_DATABASE: str = "healthpal"
    DATABASE_URL: Optional[str] = None
```

```
    # 数据库配置 - MongoDB
    MONGODB_URI: str = "mongodb://localhost:27017"
    MONGODB_DATABASE: str = "healthpal_mongo"

    # 缓存配置 - Redis
    REDIS_HOST: str = "localhost"
    REDIS_PORT: int = 6379
    REDIS_DB: int = 0
    REDIS_PASSWORD: Optional[str] = None

    # JWT 配置
    JWT_SECRET_KEY: str = "your-secret-key-change-in-production"
    JWT_ALGORITHM: str = "HS256"
    JWT_ACCESS_TOKEN_EXPIRE_MINUTES: int = 10080  # 7 天

    # 安全配置
    SECRET_KEY: str = "your-secret-key-for-password"
    ALGORITHM: str = "HS256"
    ACCESS_TOKEN_EXPIRE_MINUTES: int = 10080

    # CORS 配置
    CORS_ORIGINS: list = [
        "http://localhost:3000",
        "http://localhost:8080",
    ]

    # 第三方服务配置
    OCR_PROVIDER: str = "baidu"  # baidu/tencent
    BAIDU_OCR_API_KEY: str = ""
    BAIDU_OCR_SECRET_KEY: str = ""

    # 文件存储配置
    OSS_PROVIDER: str = "local"  # local/aliyun/tencent
    OSS_BUCKET: str = "healthpal"
    OSS_ENDPOINT: str = ""
    OSS_ACCESS_KEY: str = ""
    OSS_SECRET_KEY: str = ""

    # 通知服务配置
    SMS_PROVIDER: str = "aliyun"
    EMAIL_SMTP_HOST: str = "smtp.gmail.com"
    EMAIL_SMTP_PORT: int = 587
    EMAIL_USER: str = ""
    EMAIL_PASSWORD: str = ""
```

```
    class Config:
        env_file = ".env"
        case_sensitive = True

@lru_cache()
def get_settings() -> Settings:
    """获取配置单例"""
    return Settings()

settings = get_settings()
```

4.2 数据库模型 (models/)

用户模型 (user.py)

```
from sqlalchemy import Column, BigInteger, String, DateTime, func
from sqlalchemy.orm import relationship
from app.db.session import Base
import uuid
from datetime import datetime

class User(Base):
    """用户表"""
    __tablename__ = "users"

    id = Column(BigInteger, primary_key=True, autoincrement=True, comment="主键 ID")
    uuid = Column(String(36), unique=True, nullable=False, index=True, comment="用户 UUID")
    phone = Column(String(20), unique=True, index=True, comment="手机号")
    email = Column(String(100), unique=True, index=True, comment="邮箱")
    password_hash = Column(String(255), nullable=False, comment="密码哈希")
    nickname = Column(String(50), comment="昵称")
    avatar_url = Column(String(255), comment="头像 URL")
    gender = Column(String(1), default="0", comment="性别 0:未知 1:男 2:女")
    birthday = Column(DateTime, comment="生日")

    created_at = Column(DateTime, default=func.now(), comment="创建时间")
    updated_at = Column(DateTime, default=func.now(), onupdate=func.now(), comment="更新时间")
    deleted_at = Column(DateTime, index=True, comment="删除时间（软删除）")

    # 关联关系
    family_members = relationship("FamilyMember", back_populates="user", cascade="all, del
    health_records = relationship("HealthRecord", back_populates="user", cascade="all, del
```

```
    medication_reminders = relationship("MedicationReminder", back_populates="user", casca

    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        if not self.uuid:
            self.uuid = str(uuid.uuid4())

    def __repr__(self):
        return f"<User(uuid={self.uuid}, phone={self.phone})>"
```

健康档案模型 (health_record.py)

```
from sqlalchemy import Column, BigInteger, String, Date, DateTime, func, ForeignKey, JSON
from sqlalchemy.orm import relationship
from app.db.session import Base
import uuid

class HealthRecord(Base):
    """健康档案表"""
    __tablename__ = "health_records"

    id = Column(BigInteger, primary_key=True, autoincrement=True, comment="主键 ID")
    uuid = Column(String(36), unique=True, nullable=False, index=True, comment="档案 UUID")
    user_id = Column(BigInteger, ForeignKey("users.id"), nullable=False, index=True, commen
    member_id = Column(BigInteger, ForeignKey("family_members.id"), index=True, comment="家

    record_type = Column(String(50), nullable=False, index=True, comment="档案类型")
    title = Column(String(200), nullable=False, comment="档案标题")
    hospital = Column(String(100), comment="医院名称")
    department = Column(String(50), comment="科室")
    record_date = Column(Date, nullable=False, index=True, comment="检查日期")

    ocr_status = Column(String(20), default="pending", comment="OCR 状态: pending/processing
    ocr_result = Column(JSON, comment="OCR 识别结果（原始 JSON）")
    structured_data = Column(JSON, comment="结构化数据（提取的指标等）")
    tags = Column(JSON, comment="标签数组")
    file_urls = Column(JSON, comment="文件 URL 列表")

    created_at = Column(DateTime, default=func.now(), comment="创建时间")
    updated_at = Column(DateTime, default=func.now(), onupdate=func.now(), comment="更新时间")

    # 关联关系
    user = relationship("User", back_populates="health_records")
```

```
    member = relationship("FamilyMember", back_populates="health_records")
    indicators = relationship("HealthIndicator", back_populates="record", cascade="all, del

    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        if not self.uuid:
            self.uuid = str(uuid.uuid4())
```

4.3 Pydantic Schema (schemas/)

用户相关 Schema (user.py)

```
from pydantic import BaseModel, Field, EmailStr, field_validator
from typing import Optional
from datetime import datetime
import re

class UserBase(BaseModel):
    """用户基础 Schema"""
    nickname: Optional[str] = Field(None, max_length=50, description="昵称")
    avatar_url: Optional[str] = Field(None, max_length=255, description="头像 URL")
    gender: Optional[str] = Field("0", pattern="^[012]$", description="性别")
    birthday: Optional[datetime] = Field(None, description="生日")

class UserCreate(BaseModel):
    """用户创建 Schema"""
    phone: str = Field(..., pattern=r"^1[3-9]\d{9}$", description="手机号")
    password: str = Field(..., min_length=6, max_length=32, description="密码")
    verify_code: str = Field(..., min_length=6, max_length=6, description="验证码")
    nickname: Optional[str] = Field(None, max_length=50, description="昵称")

    @field_validator('password')
    @classmethod
    def validate_password(cls, v):
        if not re.match(r"^[a-zA-Z0-9_@#$%^&+=!~.-]+$", v):
            raise ValueError('密码只能包含字母、数字和特殊字符')
        return v

class UserLogin(BaseModel):
    """用户登录 Schema"""
    phone: str = Field(..., pattern=r"^1[3-9]\d{9}$", description="手机号")
    password: str = Field(..., description="密码")
```

```
    verify_code: Optional[str] = Field(None, description="验证码")

class UserResponse(UserBase):
    """用户响应 Schema"""
    uuid: str
    phone: str
    email: Optional[str] = None
    created_at: datetime

    class Config:
        from_attributes = True

class TokenData(BaseModel):
    """Token 数据 Schema"""
    uuid: Optional[str] = None
```

通用响应 Schema (common.py)

```
from pydantic import BaseModel, Field
from typing import Optional, Generic, TypeVar, List
from datetime import datetime

T = TypeVar('T')

class ResponseModel(BaseModel):
    """通用响应模型"""
    code: int = Field(0, description="错误码: 0 表示成功")
    message: str = Field("success", description="响应消息")
    data: Optional[dict] = Field(None, description="响应数据")

class PageResponse(BaseModel):
    """分页响应模型"""
    total: int = Field(..., description="总记录数")
    page: int = Field(..., description="当前页码")
    page_size: int = Field(..., description="每页大小")
    items: List[dict] = Field(..., description="数据列表")

class ErrorResponse(BaseModel):
    """错误响应模型"""
    code: int = Field(..., description="错误码")
    message: str = Field(..., description="错误消息")
    details: Optional[dict] = Field(None, description="详细错误信息")
```

4.4 API 路由 (api/)

认证接口 (auth.py)

```
from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordRequestForm
from sqlalchemy.ext.asyncio import AsyncSession
from app.db.session import get_db
from app.schemas.user import UserCreate, UserLogin, UserResponse, TokenData
from app.schemas.common import ResponseModel
from app.services.auth_service import AuthService
from app.dependencies import get_current_user
from app.models.user import User
import uuid

router = APIRouter(prefix="/auth", tags=["认证"])

@router.post("/register", response_model=ResponseModel, summary="用户注册")
async def register(
    user_data: UserCreate,
    db: AsyncSession = Depends(get_db)
):
    """
    用户注册接口

     - **phone**: 手机号（11 位中国大陆手机号）
     - **password**: 密码（6-32 位）
     - **verify_code**: 短信验证码
     - **nickname**: 昵称（可选）
    """
    auth_service = AuthService(db)

    # 验证短信验证码
    if not await auth_service.verify_sms_code(user_data.phone, user_data.verify_code):
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="验证码错误"
        )

    # 检查手机号是否已注册
    existing_user = await auth_service.get_user_by_phone(user_data.phone)
    if existing_user:
        raise HTTPException(
```

```
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="手机号已注册"
        )

    # 创建用户
    user = await auth_service.create_user(
        phone=user_data.phone,
        password=user_data.password,
        nickname=user_data.nickname
    )

    return ResponseModel(
        code=0,
        message="注册成功",
        data={"uuid": user.uuid}
    )

@router.post("/login", response_model=ResponseModel, summary="用户登录")
async def login(
    login_data: UserLogin,
    db: AsyncSession = Depends(get_db)
):
    """
    用户登录接口

    返回 JWT Token 和用户信息
    """
    auth_service = AuthService(db)

    # 验证用户
    user = await auth_service.authenticate_user(
        phone=login_data.phone,
        password=login_data.password
    )

    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="手机号或密码错误"
        )

    # 生成 Token
    access_token = await auth_service.create_access_token(user.uuid)

    return ResponseModel(
```

```
        code=0,
        message="登录成功",
        data={
            "token": access_token,
            "token_type": "Bearer",
            "expires_in": 604800,  # 7 天
            "user": {
                "uuid": user.uuid,
                "nickname": user.nickname,
                "avatar": user.avatar_url
            }
        }
    )

@router.get("/me", response_model=ResponseModel, summary="获取当前用户信息")
async def get_current_user_info(
    current_user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db)
):
    """获取当前登录用户的信息"""
    return ResponseModel(
        code=0,
        data={
            "uuid": current_user.uuid,
            "phone": current_user.phone,
            "email": current_user.email,
            "nickname": current_user.nickname,
            "avatar": current_user.avatar_url,
            "gender": current_user.gender,
            "birthday": current_user.birthday
        }
    )
```

健康档案接口 (records.py)

```
from fastapi import APIRouter, Depends, HTTPException, status, UploadFile, File, Form
from sqlalchemy.ext.asyncio import AsyncSession
from typing import List, Optional
from app.db.session import get_db
from app.schemas.record import RecordCreate, RecordResponse, RecordListResponse
from app.schemas.common import ResponseModel, PageResponse
from app.services.record_service import RecordService
from app.dependencies import get_current_user
```

```
from app.models.user import User

router = APIRouter(prefix="/records", tags=["健康档案"])

@router.post("/upload", response_model=ResponseModel, summary="上传健康档案")
async def upload_record(
    file: UploadFile = File(..., description="图片/PDF 文件"),
    record_type: str = Form(..., description="档案类型"),
    title: str = Form(..., description="档案标题"),
    record_date: str = Form(..., description="检查日期"),
    member_id: Optional[int] = Form(None, description="家庭成员 ID"),
    hospital: Optional[str] = Form(None, description="医院名称"),
    current_user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db)
):
    """
    上传健康档案

    支持格式: JPG, PNG, PDF
    最大大小: 10MB
    """
    # 验证文件类型
    allowed_types = ["image/jpeg", "image/png", "application/pdf"]
    if file.content_type not in allowed_types:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="不支持的文件格式"
        )

    # 验证文件大小 (10MB)
    file_size = 0
    content = await file.read()
    file_size = len(content)
    if file_size > 10 * 1024 * 1024:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="文件大小超过 10MB"
        )

    record_service = RecordService(db)

    # 创建档案记录
    record = await record_service.create_record(
        user_id=current_user.id,
        member_id=member_id,
```

```
        record_type=record_type,
        title=title,
        record_date=record_date,
        hospital=hospital
    )

    # 上传文件到 OSS
    file_service = record_service.file_service
    file_url = await file_service.upload_file(
        content=content,
        filename=file.filename,
        content_type=file.content_type,
        record_uuid=record.uuid
    )

    # 更新档案文件 URL
    await record_service.update_record_files(record.uuid, [file_url])

    # 触发 OCR 识别任务（异步）
    await record_service.trigger_ocr_task(record.uuid, file_url)

    return ResponseModel(
        code=0,
        message="上传成功",
        data={
            "record_id": record.uuid,
            "ocr_status": "pending"
        }
    )

@router.get("", response_model=ResponseModel, summary="获取档案列表")
async def get_records(
    page: int = 1,
    page_size: int = 20,
    member_id: Optional[int] = None,
    record_type: Optional[str] = None,
    current_user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db)
):
    """获取健康档案列表（分页）"""
    record_service = RecordService(db)

    records, total = await record_service.get_records(
        user_id=current_user.id,
```

```
        member_id=member_id,
        record_type=record_type,
        page=page,
        page_size=page_size
    )

    return ResponseModel(
        code=0,
        data={
            "total": total,
            "page": page,
            "page_size": page_size,
            "records": records
        }
    )

@router.get("/{record_id}", response_model=ResponseModel, summary="获取档案详情")
async def get_record_detail(
    record_id: str,
    current_user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db)
):
    """获取健康档案详情"""
    record_service = RecordService(db)

    record = await record_service.get_record_by_uuid(record_id, current_user.id)

    if not record:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="档案不存在"
        )

    return ResponseModel(
        code=0,
        data=record
    )

@router.delete("/{record_id}", response_model=ResponseModel, summary="删除档案")
async def delete_record(
    record_id: str,
    current_user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db)
):
```

```
    """删除健康档案（软删除）"""
    record_service = RecordService(db)

    success = await record_service.delete_record(record_id, current_user.id)

    if not success:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="档案不存在"
        )

    return ResponseModel(
        code=0,
        message="删除成功"
    )
```

4.5 服务层 (services/)

认证服务 (auth_service.py)

```
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select
from app.models.user import User
from app.utils.password import verify_password, get_password_hash
from app.utils.jwt import create_access_token
from app.db.redis import redis_client
import json

class AuthService:
    """认证服务类"""

    def __init__(self, db: AsyncSession):
        self.db = db

    async def get_user_by_phone(self, phone: str) -> User | None:
        """根据手机号获取用户"""
        result = await self.db.execute(
            select(User).where(User.phone == phone, User.deleted_at.is_(None))
        )
        return result.scalar_one_or_none()

    async def get_user_by_uuid(self, uuid: str) -> User | None:
        """根据 UUID 获取用户"""
```

```
        result = await self.db.execute(
            select(User).where(User.uuid == uuid, User.deleted_at.is_(None))
        )
        return result.scalar_one_or_none()

    async def create_user(
        self,
        phone: str,
        password: str,
        nickname: str | None = None
    ) -> User:
        """创建新用户"""
        password_hash = get_password_hash(password)

        user = User(
            phone=phone,
            password_hash=password_hash,
            nickname=nickname
        )

        self.db.add(user)
        await self.db.commit()
        await self.db.refresh(user)

        return user

    async def authenticate_user(self, phone: str, password: str) -> User | None:
        """验证用户登录"""
        user = await self.get_user_by_phone(phone)

        if not user:
            return None

        if not verify_password(password, user.password_hash):
            return None

        return user

    async def create_access_token(self, user_uuid: str) -> str:
        """生成访问 Token"""
        return create_access_token(data={"sub": user_uuid})

    async def verify_sms_code(self, phone: str, code: str) -> bool:
        """验证短信验证码"""
```

```
        key = f"sms:code:{phone}"
        stored_code = await redis_client.get(key)

        if not stored_code:
            return False

        return stored_code.decode() == code

    async def set_sms_code(self, phone: str, code: str, expire: int = 300):
        """设置短信验证码（5 分钟有效期）"""
        key = f"sms:code:{phone}"
        await redis_client.setex(key, expire, code)
```

档案服务 (record_service.py)

```
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select, func
from app.models.health_record import HealthRecord
from app.models.health_indicator import HealthIndicator
from app.services.file_service import FileService
from app.services.ocr_service import OCRService
from app.tasks.ocr_tasks import process_ocr_task
from typing import List, Tuple, Optional
from datetime import datetime
import json

class RecordService:
    """健康档案服务类"""

    def __init__(self, db: AsyncSession):
        self.db = db
        self.file_service = FileService()
        self.ocr_service = OCRService()

    async def create_record(
        self,
        user_id: int,
        record_type: str,
        title: str,
        record_date: str,
        member_id: Optional[int] = None,
        hospital: Optional[str] = None,
        department: Optional[str] = None
```

```
    ) -> HealthRecord:
        """创建健康档案记录"""
        record = HealthRecord(
            user_id=user_id,
            member_id=member_id,
            record_type=record_type,
            title=title,
            record_date=datetime.strptime(record_date, "%Y-%m-%d").date(),
            hospital=hospital,
            department=department,
            ocr_status="pending"
        )

        self.db.add(record)
        await self.db.commit()
        await self.db.refresh(record)

        return record

    async def get_records(
        self,
        user_id: int,
        member_id: Optional[int] = None,
        record_type: Optional[str] = None,
        page: int = 1,
        page_size: int = 20
    ) -> Tuple[List[dict], int]:
        """获取档案列表（分页）"""
        # 构建查询
        query = select(HealthRecord).where(
            HealthRecord.user_id == user_id,
            HealthRecord.deleted_at.is_(None)
        )

        if member_id:
            query = query.where(HealthRecord.member_id == member_id)

        if record_type:
            query = query.where(HealthRecord.record_type == record_type)

        # 获取总数
        count_query = select(func.count()).select_from(query.subquery())
        total = (await self.db.execute(count_query)).scalar()
```

```
        # 分页查询
        query = query.order_by(HealthRecord.record_date.desc())
        query = query.offset((page - 1) * page_size).limit(page_size)

        result = await self.db.execute(query)
        records = result.scalars().all()

        # 转换为字典列表
        records_data = [
            {
                "uuid": r.uuid,
                "title": r.title,
                "record_type": r.record_type,
                "hospital": r.hospital,
                "record_date": r.record_date.isoformat(),
                "ocr_status": r.ocr_status,
                "file_urls": r.file_urls or []
            }
            for r in records
        ]

        return records_data, total

    async def get_record_by_uuid(self, record_uuid: str, user_id: int) -> dict | None:
        """根据 UUID 获取档案详情"""
        result = await self.db.execute(
            select(HealthRecord).where(
                HealthRecord.uuid == record_uuid,
                HealthRecord.user_id == user_id,
                HealthRecord.deleted_at.is_(None)
            )
        )

        record = result.scalar_one_or_none()

        if not record:
            return None

        # 获取关联的指标
        indicators_result = await self.db.execute(
            select(HealthIndicator).where(
                HealthIndicator.record_id == record.id
            )
        )
        indicators = indicators_result.scalars().all()
```

```
    return {
        "uuid": record.uuid,
        "title": record.title,
        "record_type": record.record_type,
        "hospital": record.hospital,
        "department": record.department,
        "record_date": record.record_date.isoformat(),
        "ocr_status": record.ocr_status,
        "ocr_result": record.ocr_result,
        "structured_data": record.structured_data,
        "indicators": [
            {
                "code": i.indicator_code,
                "name": i.indicator_name,
                "value": float(i.value),
                "unit": i.unit,
                "reference_min": float(i.reference_min) if i.reference_min else None,
                "reference_max": float(i.reference_max) if i.reference_max else None,
                "status": i.status
            }
            for i in indicators
        ],
        "file_urls": record.file_urls or []
    }

async def update_record_files(self, record_uuid: str, file_urls: List[str]):
    """更新档案文件 URL"""
    result = await self.db.execute(
        select(HealthRecord).where(HealthRecord.uuid == record_uuid)
    )
    record = result.scalar_one_or_none()

    if record:
        record.file_urls = file_urls
        await self.db.commit()

async def trigger_ocr_task(self, record_uuid: str, file_url: str):
    """触发 OCR 识别任务"""
    # 更新状态为处理中
    result = await self.db.execute(
        select(HealthRecord).where(HealthRecord.uuid == record_uuid)
    )
    record = result.scalar_one_or_none()
```

```
        if record:
            record.ocr_status = "processing"
            await self.db.commit()

        # 触发异步任务
        process_ocr_task.delay(record_uuid, file_url)

    async def delete_record(self, record_uuid: str, user_id: int) -> bool:
        """软删除档案"""
        from datetime import datetime

        result = await self.db.execute(
            select(HealthRecord).where(
                HealthRecord.uuid == record_uuid,
                HealthRecord.user_id == user_id
            )
        )
        record = result.scalar_one_or_none()

        if not record:
            return False

        record.deleted_at = datetime.now()
        await self.db.commit()

        return True
```

4.6 异步任务 (tasks/)

OCR 任务 (ocr_tasks.py)

```
from celery import Celery
from app.config import settings
from app.services.ocr_service import OCRService
from app.db.session import AsyncSessionLocal
from app.models.health_record import HealthRecord
from sqlalchemy import select
import asyncio
import json

# Celery 配置
celery_app = Celery(
```

```
    'healthpal',
    broker=f"redis://{settings.REDIS_HOST}:{settings.REDIS_PORT}/{settings.REDIS_DB}",
    backend=f"redis://{settings.REDIS_HOST}:{settings.REDIS_PORT}/{settings.REDIS_DB}"
)

celery_app.conf.update(
    task_serializer='json',
    accept_content=['json'],
    result_serializer='json',
    timezone='Asia/Shanghai',
    enable_utc=True,
    task_track_started=True,
    task_time_limit=300,  # 5 分钟超时
)

@celery_app.task(bind=True, max_retries=3)
def process_ocr_task(self, record_uuid: str, file_url: str):
    """
     OCR 识别异步任务

    支持重试机制，最多重试 3 次
    """
    try:
        ocr_service = OCRService()

        # 调用 OCR 服务
        ocr_result = ocr_service.recognize(file_url)

        if not ocr_result:
            raise Exception("OCR 识别失败")

        # 解析 OCR 结果，提取结构化数据
        structured_data = ocr_service.parse_ocr_result(ocr_result)

        # 更新数据库
        asyncio.run(update_record_ocr_status(
            record_uuid,
            "completed",
            ocr_result,
            structured_data
        ))

        return {"status": "success", "record_uuid": record_uuid}

    except Exception as e:
```

```
        # 重试逻辑
        try:
            raise self.retry(exc=e, countdown=60)  # 60 秒后重试
        except self.MaxRetriesExceededError:
            # 超过最大重试次数，标记为失败
            asyncio.run(update_record_ocr_status(
                record_uuid,
                "failed",
                None,
                None,
                error=str(e)
            ))
            raise

async def update_record_ocr_status(
    record_uuid: str,
    status: str,
    ocr_result: dict,
    structured_data: dict,
    error: str = None
):
    """更新档案 OCR 状态"""
    async with AsyncSessionLocal() as session:
        result = await session.execute(
            select(HealthRecord).where(HealthRecord.uuid == record_uuid)
        )
        record = result.scalar_one_or_none()

        if record:
            record.ocr_status = status

            if ocr_result:
                record.ocr_result = ocr_result

            if structured_data:
                record.structured_data = structured_data

            await session.commit()
```

五、数据库设计

5.1 MySQL 表结构

用户表 (users)

字段	类型	说明
id	BIGINT	主键，自增
uuid	VARCHAR(36)	用户 UUID，唯一索引
phone	VARCHAR(20)	手机号，唯一索引
email	VARCHAR(100)	邮箱，唯一索引
password_hash	VARCHAR(255)	密码哈希
nickname	VARCHAR(50)	昵称
avatar_url	VARCHAR(255)	头像 URL
gender	TINYINT	性别：0 未知/1 男/2 女
birthday	DATE	生日
created_at	TIMESTAMP	创建时间
updated_at	TIMESTAMP	更新时间
deleted_at	TIMESTAMP	删除时间（软删除）

健康档案表 (health_records)

字段	类型	说明
id	BIGINT	主键，自增
uuid	VARCHAR(36)	档案 UUID

字段	类型	说明
user_id	BIGINT	用户 ID，外键
member_id	BIGINT	家庭成员 ID，外键
record_type	VARCHAR(50)	档案类型
title	VARCHAR(200)	档案标题
hospital	VARCHAR(100)	医院名称
department	VARCHAR(50)	科室
record_date	DATE	检查日期
ocr_status	VARCHAR(20)	OCR 状态
ocr_result	JSON	OCR 原始结果
structured_data	JSON	结构化数据
file_urls	JSON	文件 URL 列表
created_at	TIMESTAMP	创建时间
updated_at	TIMESTAMP	更新时间

5.2 索引设计

```
-- 用户表索引
CREATE INDEX idx_users_uuid ON users(uuid);
CREATE INDEX idx_users_phone ON users(phone);
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_deleted ON users(deleted_at);

-- 健康档案表索引
CREATE INDEX idx_records_uuid ON health_records(uuid);
CREATE INDEX idx_records_user_id ON health_records(user_id);
CREATE INDEX idx_records_member_id ON health_records(member_id);
CREATE INDEX idx_records_date ON health_records(record_date);
CREATE INDEX idx_records_type ON health_records(record_type);
```

```
CREATE INDEX idx_records_user_date ON health_records(user_id, record_date);

-- 复合索引（常用查询）
CREATE INDEX idx_records_user_type_date ON health_records(user_id, record_type, record_dat
```

5.3 Redis Key 设计

```
# 认证相关
session:{uuid}                  # 用户会话，TTL 7 天
token:blacklist:{jti}            # Token 黑名单，TTL 为 Token 剩余有效期

# 验证码
sms:code:{phone}                # 短信验证码，TTL 5 分钟
email:code:{email}              # 邮箱验证码，TTL 5 分钟

# 缓存
user:profile:{user_id}           # 用户资料，TTL 30 分钟
record:detail:{record_uuid}      # 档案详情，TTL 1 小时

# 分布式锁
lock:upload:{user_id}            # 上传锁，TTL 30 秒
lock:ocr:{record_uuid}           # OCR 处理锁，TTL 5 分钟

# 限流
ratelimit:api:{user_id}:{endpoint}  # API 限流计数，TTL 1 分钟
```

六、API 接口清单

6.1 认证接口

方法	路径	说明	认证
POST	/api/v1/auth/register	用户注册	×
POST	/api/v1/auth/login	用户登录	×
POST	/api/v1/auth/logout	用户登出	

方法	路径	说明	认证
GET	/api/v1/auth/me	获取当前用户	
PUT	/api/v1/auth/password	修改密码	

6.2 用户接口

方法	路径	说明	认证
GET	/api/v1/user/profile	获取用户资料	
PUT	/api/v1/user/profile	更新用户资料	
POST	/api/v1/user/avatar	上传头像	

6.3 家庭成员接口

方法	路径	说明	认证
GET	/api/v1/family/members	获取家庭成员列表	
POST	/api/v1/family/members	添加家庭成员	
PUT	/api/v1/family/members/{id}	更新家庭成员	
DELETE	/api/v1/family/members/{id}	删除家庭成员	

6.4 健康档案接口

方法	路径	说明	认证
POST	/api/v1/records/upload	上传档案	
GET	/api/v1/records	获取档案列表	
GET	/api/v1/records/{id}	获取档案详情	
DELETE	/api/v1/records/{id}	删除档案	

6.5 健康指标接口

方法	路径	说明	认证
GET	/api/v1/indicators/trend	获取指标趋势	
POST	/api/v1/indicators	手动添加指标	
DELETE	/api/v1/indicators/{id}	删除指标	

6.6 用药提醒接口

方法	路径	说明	认证
GET	/api/v1/medications/reminders	获取提醒列表	
POST	/api/v1/medications/reminders	创建提醒	
PUT	/api/v1/medications/reminders/{id}	更新提醒	
DELETE	/api/v1/medications/reminders/{id}	删除提醒	
POST	/api/v1/medications/reminders/{id}/confirm	确认用药	

6.7 AI 分析接口

方法	路径	说明	认证
GET	/api/v1/ai/analysis/report	获取健康分析报告	
POST	/api/v1/ai/analysis/risk	健康风险评估	

七、安全设计

7.1 认证安全

- **密码存储**: bcrypt 加密（cost factor=12）
- **Token 机制**: JWT + Refresh Token
- **Token 有效期**: Access Token 7 天
- **Token 黑名单**: Redis 存储已注销 Token

7.2 接口安全

- **HTTPS**: 强制 TLS 1.3
- **CORS**: 严格限制来源
- **限流**: Redis 计数器，防止滥用
- **SQL 注入防护**: SQLAlchemy ORM 参数化查询
- **XSS 防护**: 输入验证 + 输出转义

7.3 数据安全

- **敏感数据脱敏**: 手机号中间 4 位隐藏
- **文件存储**: 私有 Bucket + 签名 URL
- **数据备份**: 每日自动备份

八、部署配置

8.1 Docker Compose (开发环境)

```
version: '3.8'

services:
  mysql:
```

```
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: root_password
      MYSQL_DATABASE: healthpal
      MYSQL_USER: healthpal
      MYSQL_PASSWORD: your_password
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql

  mongodb:
    image: mongo:6.0
    ports:
      - "27017:27017"
    volumes:
      - mongodb_data:/data/db

  redis:
    image: redis:7.0-alpine
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data

  backend:
    build: ./src/backend
    ports:
      - "8000:8000"
    environment:
      - MYSQL_HOST=mysql
      - MONGODB_URI=mongodb://mongodb:27017
      - REDIS_HOST=redis
    depends_on:
      - mysql
      - mongodb
      - redis
    volumes:
      - ./src/backend:/app
    command: uvicorn app.main:app --reload --host 0.0.0.0 --port 8000

volumes:
  mysql_data:
```

```
  mongodb_data:
  redis_data:
```

8.2 生产环境部署

- **容器编排**: Kubernetes
- **负载均衡**: Nginx Ingress
- **监控**: Prometheus + Grafana
- **日志**: ELK Stack
- **CI/CD**: GitHub Actions

九、开发规范

9.1 代码规范

- **类型注解**: 强制使用 Python Type Hints
- **文档字符串**: 所有公共函数必须有 docstring
- **错误处理**: 使用自定义异常类
- **日志**: 使用 structlog 结构化日志

9.2 Git 规范

```
feat: 新功能
fix: 修复 bug
docs: 文档更新
style: 代码格式
refactor: 重构
test: 测试相关
chore: 构建/工具
```

9.3 测试规范

- **单元测试**: pytest + pytest-asyncio
- **覆盖率**: 目标 80%+
- **集成测试**: TestContainer

文档结束